

Non-Linear Budget Constraints

In-kind subsidies, partial subsidies and exclusive schemes. Each one bends the budget line somewhere different, and the optimum can land right on the fold.

BEFORE YOU START

Opening it

There is nothing to install. It is a web page: it opens in Chrome, Safari, Edge or Firefox, on a computer, tablet or phone. You need a connection the first time; after that the browser keeps it and it works offline.

Top right there are two pairs of buttons. The first switches the language between **ES** and **EN**; the second goes from **Dark** to **Light**. For projecting in class, light mode reads better on a beamer; on screen, dark is easier on the eyes.

Every graph zooms. Mouse wheel over the graph, or a two-finger pinch on the trackpad or touchscreen. The zoom applies to the graph under the pointer only, not to the whole page.

Each slider has a number box beside it. For an exact value, type it into the box and press **Enter**; the slider is for exploring, the box is for precision.

WHAT IS ON SCREEN

Up to three panels

PANEL	WHAT IT SHOWS
Plane (x, y)	The budget frontier with its folds, the feasible set, the indifference curve through the optimum and — if you ask — a second scheme overlaid for comparison.
Utility along the constraint	Utility traced along the frontier, from $x = 0$ to the far end. The optimum is this curve's maximum, and here you can see why it sometimes lands on a sharp peak.
3D surface · $z = u(x, y)$	Optional, at the foot of the screen. The same constraint lifted onto the utility surface: the optimum is the summit of the arch.

The readouts

READOUT	HOW TO READ IT
$x^*, y^*, u(x^*, y^*)$	The optimal bundle and its utility.
Cost	What the programme costs whoever funds it.
Cash advantage	How much more utility a cash transfer of the same cost would reach. This is the number that carries the argument.
Kind of optimum	Tangency, kink, corner or jump edge. Each needs a different argument, and only the first admits “MRS = relative price”.
Slope at x_s	How much the frontier's slope changes as you cross the allowance.
Jump at x_s	How far the frontier drops vertically at the allowance. Zero except in the exclusive schemes.

THE SEVEN SCHEMES

What each one does

All of them start from the same income m and the same prices. What changes is where and how the line bends. The allowance is called x_s and the subsidy intensity s .

SCHEME	THE FRONTIER IT PRODUCES
Simple constraint	The ordinary line, no programme. The benchmark.
In-kind subsidy (non-exclusive)	The first x_s units of x are free. The frontier starts flat up to x_s and from there falls with the normal slope. Cost: $p_x \cdot x_s$.
Partial in-kind subsidy	A discount of s on the first x_s units. The frontier starts with slope $(1-s) \cdot p_x/p_y$ and then reverts to normal. Cost: $s \cdot p_x \cdot x_s$.
Exclusive subsidy (public schools)	Like the in-kind one, but going past the allowance forfeits the whole grant . The frontier drops vertically at x_s and continues from the original line. The standard example is public schooling: enrol elsewhere and you lose the place.
Partial exclusive subsidy	The same with a discount instead of free units. The drop is smaller but it is there.
Cash equivalent (full)	A cash transfer of $p_x \cdot x_s$. The line shifts outward, parallel, with no bend.
Cash equivalent (partial)	The same with $s \cdot p_x \cdot x_s$. The fair rival to the partial subsidy.

The **s** slider reaches down to -1 . With s negative the programme stops being a subsidy and becomes a **tax** at rate $\tau = |s|$ on the first x_s units: the frontier bends inward instead of outward.

Compare with

The **Compare with** menu overlays a second scheme in another colour. The comparison that matters is nearly always against the cash transfer of equal cost, and that is the default.

THE CLASSIC ARGUMENT, IN NUMBERS

Cash versus in-kind

The standard result is that a cash transfer is never worse for the recipient than an in-kind subsidy of the same cost, and is sometimes strictly better. You can check that in two minutes [here](#) and — more importantly — see *when* there is a difference and when there is not.

- 1 Set **Cobb-Douglas**, $p_x = 2$, $p_y = 2$, $m = 20$, $x_s = 4$. Pick the scheme **In-kind subsidy (non-exclusive)** and compare with the **Cash equivalent (full)**.

Programme cost: $p_x \cdot x_s = 8$.

- 2 Leave $a = 0.5$. Look at the two optima.

Both give $x^* = 7$, $y^* = 7$, $u = 7$. **A tie**. The optimum lands where the two frontiers coincide, so the allowance does not bind.

- 3 Now lower a to **0.15**. The consumer stops wanting so much x .

In kind: $x^* = 4$, $y^* = 10$, $u = 8.716$, and the applet classes it as a **kink**. In cash: $x^* = 2.1$, $y^* = 11.9$, $u = 9.174$. **Cash wins**.

- 4 That is the whole moral: an in-kind subsidy only hurts when it *pushes*. At $a = 0.5$ the consumer already wanted 7 units and the 4 free ones force nothing; at $a = 0.15$ he only wanted 2.1, and the programme parks him at 4.

What exclusion adds

Exclusive schemes are worse still, and for a different reason. At the same cost they do not merely bend the frontier: they **break** it.

- 1 Go back to $a = 0.5$ and change the scheme to **Exclusive subsidy**, with the same $p_x = 2$, $p_y = 2$, $m = 20$, $x_s = 4$.

$x^* = 4$, $y^* = 10$, $u = 6.325$. The optimum type is **jump edge**.

- 2 Compare it with the non-exclusive in-kind subsidy, which cost the same (8) and gave $u = 7$.

The same public money, 0.675 less utility. The only thing that changed is the exclusion rule.

- 3 Switch on the **Kinks** layer and look at the **Utility along the constraint** panel: the curve drops at x_s . The consumer sticks to the left edge of the drop because crossing it costs the whole grant.

This is the applet's policy reading: it is not only how much is spent, but what shape of frontier the recipient is left with. A badly designed programme can spend the same and be worth less.

What it is for

The **3D surface** switch opens a panel at the foot with the same utility function drawn in height, and the budget frontier lifted onto it. The optimum, a tangency point in the plane, is here simply the summit of the arch: for many people that is the picture that makes it click.

CONTROL	WHAT IT DOES
3D	Free perspective view. Drag to rotate the camera, wheel to zoom.
X-Z	Orthogonal projection onto the plane $y = 0$. Utility against x alone.
X-Y	Projection onto $z = 0$: the view from above. Literally the contour map, and useful for showing where contours come from.
Y-Z	Projection onto $x = 0$: utility against y .
Shade outside the feasible set	Washes out the colour of the part of the surface you cannot afford. Only the affordable slice stays lit, which is where the problem lives.
XY plane	Draws the floor $z = 0$ and projects the frontier onto it, so you see the lifted curve and its shadow at once.

The three projections are **orthographic**, not perspective: nothing foreshortens, so distances compare directly. The free view is perspective, which reads better as a solid but misleads about measurements.

AXES AND LAYERS

Setting the axes

By default the axes are **fixed** at the ceilings you type, so two schemes compare at the same scale. If the frontier leaves the frame a warning appears: raise the ceiling or tick **Fit the axes automatically**.

The layers

LAYER	WHAT IT DRAWS
Heat gradient	The coloured utility map. Turn it off and the background goes plain; the indifference curves keep their height colour.
Contours	Indifference curves at regular levels.
Feasible set	Darkens everything outside the budget.
Curve through the optimum	The indifference curve through the solution.
Optimum	The point. Turn it off while the student looks for it.
Vertices	Marks the folds and the jumps of the frontier.
Compared set	The feasible area of the second scheme.
Grid	The grid.

WRITING YOUR OWN FUNCTION

The syntax it accepts

In the custom preferences box the variables are **x** and **y**. The optimum is found by walking the frontier piece by piece and also testing both ends of every piece, so kinks and jumps are found properly even where the first-order condition says nothing.

CONTROL	WHAT IT DOES
<code>+ - * / ^</code>	Add, subtract, multiply, divide, power. <code>x^0.5</code> is the square root of x.
<code>()</code>	Brackets. <code>(x+y)^2</code> is not the same as <code>x+y^2</code> .
<code>sqrt ln log exp abs</code>	Root, natural log, base-10 log, exponential and absolute value.
<code>min max</code>	Two arguments: <code>min(x,y)</code> is perfect complements.
<code>sin cos</code>	Trigonometric, in radians. Rarely useful here, but available.
<code>e pi</code>	The two constants. <code>e</code> is 2.718..., <code>pi</code> is 3.1416...
<code>2x</code> , <code>3xy</code>	Implicit multiplication works: <code>2x</code> reads as <code>2*x</code> .

Examples that work

EXPRESSION	WHAT PREFERENCES IT DESCRIBES
$x^{0.3}y^{0.7}$	Cobb–Douglas weighted towards y: the x allowance tends to bind.
$\min(x, y)$	Perfect complements. The optimum jumps from fold to fold.
$x+2y$	Perfect substitutes: corner solutions, and the subsidy can flip them.
$\ln(x)+y$	Quasilinear: demand for x does not depend on income, so the allowance shows clean.

If it cannot be read you get “*I can't read that expression*” and the graph stays as it was. The usual causes are an unclosed bracket, a decimal comma instead of a point (0,5 instead of 0.5) or a variable other than x and y.

IF SOMETHING GOES WRONG

Common problems

SYMPTOM	WHAT TO DO
It says “The frontier runs outside the frame”	Raise the x or y ceiling, or tick Fit the axes automatically . With a large x_s the frontier stretches a long way.
The two schemes look identical	That is a result, not a bug: the allowance is not biting. Lower the preference parameter a, or raise x_s , until the optimum lands on the fold.
The cash advantage comes out zero	Same thing: the optimum is on the stretch where the two frontiers coincide. That is exactly the case where giving in kind costs nothing in welfare.
The 3D panel is slow	Close the substitution panel, or lower the axis ceilings. The mesh is rebuilt as you rotate.
I cannot get back to the free view	Press the 3D button in the view group, or just drag on the panel: dragging leaves the projection.

Open the applet: **Non-Linear Budget Constraints**. It all runs in the browser, and once loaded, offline too.

← [Back to the menu](#)